

程序报告

学号：2312331

姓名：王一诺

一、问题重述

(简单描述对问题的理解，从问题中抓住主干，必填)

1.1 实验背景

本实验采用特征脸 (Eigenface) 算法进行人脸识别。

特征脸 (eigenface) 是第一种有效的人脸识别方法，通过在一大组描述不同人脸的图像上进行主成分分析 (PCA) 获得。本次实验要求大家构建一个自己的人脸库 (建议)：大家可以选择基于 ORL 人脸库添加自己搜集到的人脸图像形成一个更大的人脸库，要求人脸库中的每一张图像都只包含一张人脸且眼睛的中心位置对齐 (通过裁剪或缩放，使得每张人脸图像大小尺寸一致且人脸眼睛的中心位置对齐)。为了方便同学们操作，大家也可以选择直接基于 ORL 人脸库进行本次实验。

1.2 实验内容

在模型训练过程中，首先要根据测试数据求出平均脸，然后将前 K 个特征脸保存下来，利用这 K 个特征脸对测试人脸进行识别，此外对于任意给定的一张人脸图像，可以使用这 K 个特征脸对原图进行重建。

1.3 实验要求

- 1> 求解人脸图像的特征值与特征向量构建特征脸模型
- 2> 利用特征脸模型进行人脸识别和重建，比较使用不同数量特征脸的识别与重建效果
- 3> 使用 Python 语言

二、设计思想

(所采用的方法，有无对方法加以改进，该方法有哪些优化方向 (参数调整，框架调整，或者指出方法的局限性和常见问题)，伪代码，理论结果验证等... 思考题，非必填)

项目目标为：基于人脸图像数据，利用 PCA 原理提取人脸的主要特征向量 (即“特征脸”)，实现高效的人脸识别与重建。为了实现目标功能，设计以下模块：

1. spilt_data —— 数据集划分：通过数组切片和 reshape，保持图像扁平化为向量；将人脸图像数据按人和照片数量划分为训练集和测试集。
2. eigen_train —— 特征脸训练 (PCA 主成分分析)：利用 PCA 降维原理 (通过协方差矩阵求解特征向量，提取最大方差方向)，通过计算平均脸 avg_img、图像中心化 norm_img、简化特征值分解计算量、得到 K 个单位归一化的主特征向量 feature 四步提取训练集中主要的 K 个特征脸，用于表示和重建任意人脸。此外，为提升性能，用了简化协方差矩阵 $L=AA^T$ ，再映射回高维。
3. rep_face —— 特征脸投影表示：利用线性代数中的向量投影——把图像减去平均脸后投影到前 K 个特征脸上，将任意图像投影到特征脸空间，得到其低维表示向量 (特征系数)。
4. recFace —— 图像重建：利用线性组合重建公式 $\text{reconstruct} = \text{avg} + \sum c_i * u_i$ ，其中 c_i 是投影系数， u_i 是特征脸，将图像在特征脸空间中的表示向量反向变换，重建原始图像。

三、代码内容

(能体现解题思路的主要代码, 有多个文件或模块可用多个"===="隔开, 必填)

====数据预处理=====

```
# 导入必要的包
import matplotlib.pyplot as plt
import numpy as np
import cv2
from PIL import Image
import os

def spilt_data(nPerson, nPicture, data, label):
    # 数据集大小和意义
    allPerson, allPicture, rows, cols = data.shape

    # 划分训练集和测试集
    train = data[:nPerson,:nPicture,:].reshape(nPerson*nPicture, rows*cols)
    train_label = label[:nPerson, :nPicture].reshape(nPerson * nPicture)
    test = data[:nPerson, nPicture:, :, :].reshape(nPerson*(allPicture - nPicture), rows*cols)
    test_label = label[:nPerson, nPicture:].reshape(nPerson * (allPicture - nPicture))

    # 返回: 训练集, 训练集标签, 测试集, 测试集标签
    return train, train_label, test, test_label

def plot_gallery(images, titles, n_row=3, n_col=5, h=112, w=92): # 3 行 4 列
    # 展示图片
    plt.figure(figsize=(1.8 * n_col, 2.4 * n_row))
    plt.subplots_adjust(bottom=0, left=.01, right=.99, top=.90, hspace=.35)
    for i in range(n_row * n_col):
        plt.subplot(n_row, n_col, i + 1)
        plt.imshow(images[i].reshape((h, w)), cmap=plt.cm.gray)
        plt.title(titles[i], size=12)
        plt.xticks(())
        plt.yticks(())
    plt.show()
```

====特征人脸算法=====

```
def eigen_train(trainset, k=20):
    # 计算平均脸
    avg_img = np.mean(trainset, axis=0)
    # 中心化图像 (每张脸减去平均脸)
    norm_img = trainset - avg_img
    # 构造协方差矩阵的简化形式
    A = norm_img # A 的形状是 (num_samples, num_pixels)
    L = np.dot(A, A.T) # L 的形状是 (num_samples, num_samples), 计算更快
```

```

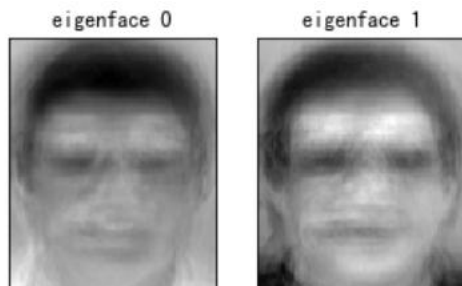
# 计算 L 的特征值和特征向量
eigenvalues, eigenvectors_small = np.linalg.eigh(L)
#从小到大排序，取最大的 k 个特征值对应的特征向量
idx = np.argsort(eigenvalues)[::-1] # 从大到小
eigenvectors_small = eigenvectors_small[:, idx[:k]] # 取前 k 个特征向量
#还原到原始高维空间的特征向量（特征脸）
eigenfaces = np.dot(A.T, eigenvectors_small)
# 归一化每个特征脸向量
eigenfaces = eigenfaces / np.linalg.norm(eigenfaces, axis=0)
feature = eigenfaces.T
# 返回：平均人脸、特征人脸、中心化人脸
return avg_img, feature, norm_img
return label, proba

# 返回平均人脸、特征人脸、中心化人脸
avg_img, eigenface_vects, trainset_vects = eigen_train(train_vectors, num_eigenface)

# 打印两张特征人脸作为展示
eigenfaces = eigenface_vects.reshape((num_eigenface, 112, 92))
eigenface_titles = ["eigenface %d" % i for i in range(eigenfaces.shape[0])]
plot_gallery(eigenfaces, eigenface_titles, n_row=1, n_col=2)

```

输出的两张特征人脸如下所示：



人脸识别模型

```

def rep_face(image, avg_img, eigenface_vects, numComponents = 0):
    # 如果未指定数量，使用全部特征脸
    if numComponents <= 0 or numComponents > eigenface_vects.shape[0]:
        numComponents = eigenface_vects.shape[0]
    # 截取前 numComponents 个特征脸
    selected_eigenfaces = eigenface_vects[:numComponents]
    numEigenFaces = numComponents
    # 将图像中心化（减去平均脸）
    norm_image = image - avg_img
    # 投影：将中心化后的图像投影到特征脸空间中
    representation = np.dot(selected_eigenfaces, norm_image)
    # 返回：输入数据的特征向量表示，特征脸使用数量

```

```

        return representation, numEigenFaces

train_reps = []
for img in train_vectors:
    train_rep, _ = rep_face(img, avg_img, eigenface_vects, num_eigenface)
    train_reps.append(train_rep)

num = 0
for idx, image in enumerate(test_vectors):
    label = test_labels[idx]
    test_rep, _ = rep_face(image, avg_img, eigenface_vects, num_eigenface)

    results = []
    for train_rep in train_reps:
        similarity = np.sum(np.square(train_rep - test_rep))
        results.append(similarity)
    results = np.array(results)

    if label == np.argmin(results) // 5 + 1:
        num = num + 1

print("人脸识别准确率: {}".format(num / 80 * 100))

```

此处使用特征脸算法对测试集中的人脸照片进行预测，定义了 `rep_face` 函数，其输入是测试数据、训练集的平均人脸数据、特征脸向量、选用的特征脸数量，最后输出人脸识别的准确率，

达到了 **91.25%**： 人脸识别准确率：91.25%

=====人脸重建模型=====

```

def recFace(representations, avg_img, eigenVectors, numComponents, sz=(112,92)):
    # 使用前 numComponents 个特征脸
    selected_eigenfaces = eigenVectors[:numComponents]
    # 从特征空间重建原始图像：重建 = 平均脸 + 特征脸线性组合
    face = avg_img + np.dot(representations, selected_eigenfaces)
    # 重建图像 reshape 回原始尺寸
    face = face.reshape(sz)
    # 返回：重建人脸, str 使用的特征人脸数量
    return face, 'numEigenFaces_{}'.format(numComponents)

print("重建训练集人脸")
# 读取 train 数据
image = train_vectors[100]
faces = []
names = []
# 选用不同数量的特征人脸重建人脸

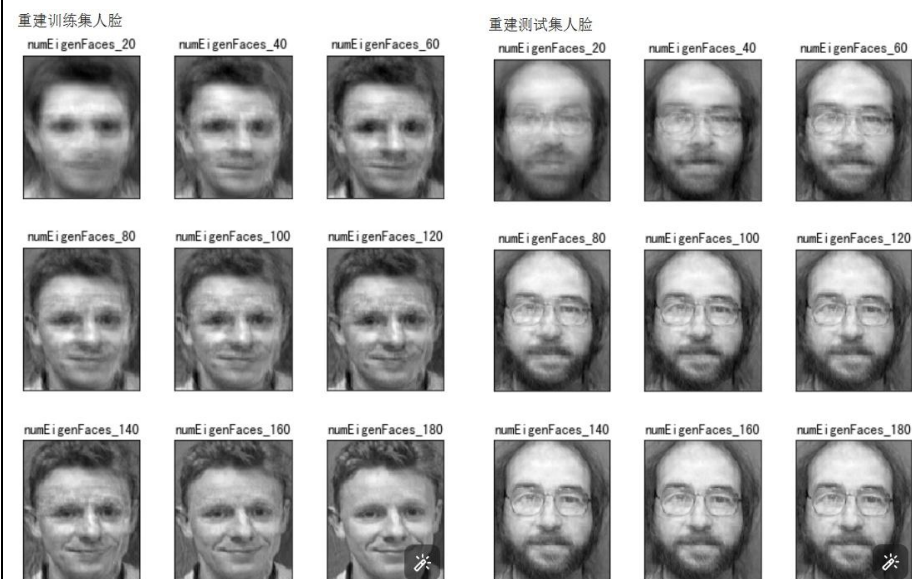
```

```

for i in range(20, 200, 20):
    representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)
    faces.append(face)
    names.append(name)
plot_gallery(faces, names, n_row=3, n_col=3)
print("-"*55)
print("重建测试集人脸")
# 读取 test 数据
image = test_vectors[54]
faces = []
names = []
# 选用不同数量的特征人脸重建人脸
for i in range(20, 200, 20):
    representations, numEigenFaces = rep_face(image, avg_img, eigenface_vects, i)
    face, name = recFace(representations, avg_img, eigenface_vects, numEigenFaces)
    faces.append(face)
    names.append(name)
plot_gallery(faces, names, n_row=3, n_col=3)

```

对人脸重建进行测试，发现比较符合原图像，如下图所示：



四、实验结果

(实验结果，必填)

将代码综合成 main 文件提交测试，提交结果如下：

接口测试

✔ 接口测试通过。

用例测试

测试点	状态	时长	结果
测试结果	✔	4s	测试成功!

提交结果

可见结果正确，人脸识别功能成功实现。

五、总结

（自评分析（是否达到目标预期，可能改进的方向，实现过程中遇到的困难，从哪些方面可以提升性能，模型的超参数和框架搜索是否合理等），**思考题，非必填**）

本次实验是对所学知识的一次综合，也是我首次处理非语言的其他模态的问题，是一次全新的体验。一个人脸识别系统通常包括人脸检测、人脸配准（对齐）、人脸表示（编码）和人脸匹配四个基本环节。本次实验采用了 Eigenfaces 算法——基于主成分分析（PCA）的算法，通过对人脸图像进行 PCA 处理，将高维的人脸数据转换为低维数据，提取出人脸的主要特征。通过各模块的实现与最终综合，在测试样例中得到了 91.25%的准确率。